

Proyecto 1: Análisis del rendimiento estudiantil en el Perú

Curso: Introducción a Machine Learning **Unidad:** Manipulación y exploración de datos con NumPy y Pandas

Modalidad: En grupos **Entregable:** `analisis.ipynb` + `reporte.csv`

1. Planteamiento del problema

La Oficina de Bienestar y Calidad Académica de una universidad con sedes y convenios en todo el país necesita entender por qué el rendimiento de sus estudiantes de primer ciclo varía tanto entre regiones. Hasta hoy, la información existe pero **nadie la ha analizado**: solo se dispone de un archivo plano con el registro de 200 estudiantes (nombre, edad, nota final y departamento de procedencia) exportado del sistema académico.

La Oficina no puede diseñar programas de tutoría ni asignar becas de forma justa mientras no responda preguntas básicas: ¿en qué departamentos se concentra la desaprobación?, ¿el rendimiento depende de la edad de ingreso?, ¿cuántos estudiantes están realmente en riesgo académico?, ¿qué nota de corte deja dentro al 10 % con mejor desempeño?

Además, el archivo **no está limpio**: durante la migración del sistema algunos registros perdieron la nota o la edad. Cualquier conclusión que se saque ignorando esos vacíos será incorrecta.

Tu rol: eres el analista de datos contratado por esa oficina. Debes transformar un archivo crudo en un reporte que un director —que no sabe programar— pueda leer y usar para tomar decisiones.

¿Por qué este proyecto importa para el curso?

Antes de entrenar cualquier modelo de machine learning es indispensable **cargar, limpiar, filtrar, agrupar y resumir** los datos. Ese trabajo representa la mayor parte del tiempo real de un proyecto de ML. Aquí practicarás exactamente eso, con las dos librerías que usarás durante todo el curso:

- **NumPy** → cálculo numérico vectorizado sobre arreglos (estadísticos, normalización, condiciones).
- **Pandas** → estructuras tabulares, limpieza, filtrado, agrupamiento y exportación.

Objetivos de aprendizaje

Al terminar el proyecto el estudiante será capaz de:

1. Leer y explorar un archivo CSV con Pandas (`read_csv`, `head`, `info`, `describe`, `shape`, `dtypes`).
 2. Detectar y tratar valores faltantes justificando la estrategia elegida (`isna`, `dropna`, `fillna`).
 3. Calcular estadísticos descriptivos con NumPy sobre arreglos (`mean`, `median`, `std`, `percentile`, `ptp`).
 4. Filtrar y seleccionar datos con condiciones lógicas (`loc`, máscaras booleanas, `query`).
 5. Crear variables derivadas con `np.where`, `np.select` y `pd.cut` (base del *feature engineering*).
 6. Agrupar y resumir información con `groupby`, `agg`, `value_counts` y `crosstab`.
 7. Estandarizar y normalizar variables numéricas, e identificar valores atípicos con el z-score.
 8. Exportar resultados reproducibles a un archivo CSV (`to_csv`).
-

2. Descripción del conjunto de datos

Archivo: `estudiantes.csv` — 200 filas, 4 columnas.

Columna	Tipo	Descripción
Nombre	texto	Nombre y apellido del estudiante
Edad	numérico	Edad en años (16 a 30). Contiene 3 valores faltantes
Nota	numérico	Nota final en escala vigesimal de 0 a 20. Contiene 6 valores faltantes
Departamento	texto	Departamento de procedencia (los 25 del Perú, incluido Callao)

Criterio institucional: se considera **aprobado** al estudiante con `Nota >= 11`.

3. Ejercicios

Son **20 ejercicios de nivel básico**, ordenados de menor a mayor dificultad. Cada uno se apoya en el anterior, así que resuélvelos en orden: del ejercicio 6 en adelante se trabaja siempre con el DataFrame ya limpio.

Bloque A — Cargar y explorar (P1–P5)

P1. Carga y primeras filas. Lee `estudiantes.csv` con `pd.read_csv` y guárdalo en una variable `df`. Muestra las primeras 10 filas con `head()` y las últimas 5 con `tail()`. ¿Qué representa cada fila del archivo?

P2. Tamaño y tipos de dato. Muestra el número de filas y columnas (`shape`), la lista de columnas (`columns`) y el tipo de dato de cada una (`dtypes`). Complementa con `info()`. ¿Qué tipo de dato tiene `Edad` y por qué no es entero, si nadie tiene una edad con decimales?

P3. Resumen estadístico general. Aplica `describe()` al DataFrame. Interpreta en dos o tres líneas lo que ves en `Nota`: ¿la media está por encima o por debajo del criterio de aprobación?, ¿qué tan dispersos están los datos?

P4. Estudiantes por departamento. ¿Cuántos estudiantes hay por departamento (`value_counts`)? ¿Cuántos departamentos distintos están representados (`nunique`)? Indica cuál concentra la mayor cantidad y qué porcentaje del total representa.

P5. Detección de valores faltantes. Determina cuántos valores faltan en cada columna (`isna().sum()`) y qué porcentaje del total representan. Muestra las filas afectadas. ¿Qué pasaría si calcularas el promedio ignorando este problema?

Bloque B — Limpiar y describir (P6–P9)

P6. Limpieza del conjunto de datos. Aplica y **justifica por escrito** esta estrategia:

- las filas sin `Nota` se eliminan (es la variable que se desea analizar),
- los valores faltantes de `Edad` se imputan con la mediana.

Verifica que ya no queden faltantes y reporta cuántas filas sobrevivieron. Guarda el resultado en `df_limpio` y **trabaja con él a partir de aquí**.

P7. Estadísticos descriptivos con NumPy. Convierte `Nota` y `Edad` a arreglos de NumPy (`.to_numpy()`) y calcula para cada uno: media (`np.mean`), mediana (`np.median`), desviación estándar (`np.std`) y varianza (`np.var`).

P8. Rango, cuartiles y el efecto de la limpieza. Sobre los mismos arreglos calcula mínimo, máximo, rango (`np.ptp`) y los cuartiles Q1, Q2 y Q3 (`np.percentile`). Luego compara la media de `Nota` del DataFrame limpio con la que da `np.nanmean` sobre los datos crudos y explica por qué coinciden o difieren.

P9. Extremos. Identifica al estudiante con la nota más alta y al de la nota más baja (nombre, edad y departamento). Considera la posibilidad de empates: si varios comparten la nota máxima, lístalos a todos. Pista: una máscara booleana (`df_limpio["Nota"] == maximo`) resuelve los empates; `idxmax()` no.

Bloque C — Filtrar y derivar variables (P10–P13)

P10. Aprobados y desaprobados. Usando una máscara booleana con el criterio `Nota >= 11`, cuenta cuántos estudiantes aprobaron y cuántos no, y calcula la **tasa global de aprobación** en porcentaje.

P11. Variable Aprobado. Crea una nueva columna `Aprobado` con valores "Sí"/"No" usando `np.where` y el mismo criterio. Verifica con `value_counts()` que coincide con lo que contaste en P10.

P12. Filtrado con condiciones múltiples. Lista a los estudiantes menores de 20 años con nota mayor o igual a 15, e indica cuántos son. Replica el resultado usando `query()` y comprueba que ambas vías devuelven lo mismo. Recuerda que en Pandas se usa `&` (no `and`) y que cada condición va entre paréntesis.

P13. Ordenamiento y Top 10. Muestra los 10 estudiantes con mayor nota (nombre, edad, nota, departamento). Resuélvelo de dos formas: con `sort_values` y con `nlargest`. ¿Cuál te parece más legible y por qué?

Bloque D — Agrupar y resumir (P14–P18)

P14. Promedio de notas por departamento. Calcula el promedio de notas por departamento con `groupby`, redondeado a dos decimales, y ordénalo de mayor a menor. Señala el mejor y el peor promedio. ¿Son comparables entre sí? Justifica mirando cuántos estudiantes tiene cada uno.

P15. Aprobados y desaprobados por departamento. Construye una tabla con el número de aprobados, desaprobados, el total y la tasa de aprobación (%) de cada departamento, ordenada de mayor a menor tasa. Pista: `groupby + agg`, o una tabla de conteo sobre la columna `Aprobado`.

P16. Grupos etarios. Usa `pd.cut` para clasificar a los estudiantes en los rangos `16-19`, `20-22`, `23-25` y `26+` en una columna nueva `GrupoEtario`. Calcula la cantidad de estudiantes, la nota promedio y la mediana de cada grupo. ¿Existe relación entre edad y rendimiento?

P17. Escala literal. Con `np.select`, convierte la nota numérica a la escala literal del sistema educativo peruano en una columna `Escala`: `AD` (18-20), `A` (14-17), `B` (11-13), `C` (0-10). Muestra cuántos estudiantes hay en cada categoría y qué porcentaje representan.

P18. Tabla cruzada. Construye una tabla cruzada (`pd.crosstab`) entre `GrupoEtario` y `Aprobado`, con totales por fila y columna (`margins=True`). Muéstrala también en porcentajes por fila (`normalize="index"`). ¿Qué grupo etario aprueba proporcionalmente más?

Bloque E — Estandarizar y entregar (P19–P20)

P19. Normalización, estandarización y valores atípicos. Sobre la columna **Nota**, calcula con NumPy:

- la normalización **min-max** al rango [0, 1] en una columna **Nota_minmax**,
- el **z-score** (restar la media y dividir entre la desviación estándar) en una columna **Nota_z**.

Lista a los estudiantes con $|z| > 2$ e interpreta qué significa estar fuera de ese rango. Explica en una línea para qué sirve escalar variables antes de entrenar un modelo de ML.

P20. Reporte consolidado y exportación. Con **groupby** y **agg**, construye una tabla final por departamento que contenga: número de estudiantes, nota promedio, mediana, desviación estándar, mínimo, máximo, aprobados, desaprobados y tasa de aprobación. Ordénala por tasa de aprobación y luego **filtra únicamente los departamentos con al menos 5 estudiantes**; explica por qué ese filtro es necesario antes de comparar regiones. Finalmente exporta la tabla a **reporte.csv** sin la columna de índice y con codificación UTF-8, y vuelve a leerla con **read_csv** para verificar que se escribió correctamente.

4. Criterios de evaluación

Criterio	Peso
El script se ejecuta de principio a fin sin errores	20 %
Uso correcto y pertinente de métodos de NumPy y Pandas (no bucles innecesarios)	25 %
Corrección de los resultados numéricos	25 %
Justificación escrita de las decisiones (limpieza, filtros, interpretación)	20 %
Orden, comentarios y legibilidad del código	10 %